

Step 01 - Get the Notebook Ready

A First Practical Machine Learning Workflow

What this document is for

This first step is deliberately not a machine-learning lesson yet. Its job is to make sure the notebook, environment, and working rules are ready before Step 02 starts creating data. The previous version tried to explain too much too early; this version keeps Step 01 focused on getting a clean starting point.

By the end of this step, you should have:

- A notebook created in the repo, with a clear title cell.
- A kernel that runs simple Python code.
- The four packages used later - pandas, NumPy, matplotlib, and scikit-learn - confirmed as available.

1. Where this step sits

The whole exercise is a bridge between prerequisite theory and actual notebook work. The later steps will build a tiny dataset, split it, train regression and classification models, measure error, provoke overfitting, and then introduce scaling, pipelines, and cross-validation.

Step	Purpose
01 - Get the notebook ready	Create the notebook, confirm the kernel, and check dependencies.
02 - Create a tiny dataset	Represent examples as rows and features as columns.
03 - Split before training	Separate training data from test data before fitting a model.
04-07 - Regression and overfitting	Train a simple regression model, measure error, and see how overfitting shows up.
08-12 - Classification and safer workflows	Move to categories, confusion matrices, scaling, pipelines, and cross-validation.

The important design rule

Nothing appears before it is explained. That includes imports, model classes, metrics, and notation. If a later document breaks that rule, fix the document rather than trying to fight through it.

2. Create the notebook

Create a new notebook in your repo. The name below avoids the old "ML loop" wording and describes what the notebook actually is: a first practical workflow.

Suggested notebook path

```
notebooks/09_first_practical_machine_learning_workflow.ipynb
```

Why this matters

The notebook is the working copy of the code-along. The PDFs explain each step, but the learning happens when you run the cells yourself and inspect the output.

Add this as the first Markdown cell:

Markdown cell

```
# A First Practical Machine Learning Workflow

## Step 01 - Get the notebook ready

This notebook will be built one small cell at a time.
I will not import, train, score, or transform anything until I know why I am doing it.
```

3. Confirm that the notebook runs

Why this cell exists

Before checking packages or writing ML code, confirm that the selected kernel can run ordinary Python. If this cell fails, the problem is the notebook/kernel setup rather than machine learning.

Code cell

```
print("Notebook is running.")
```

What to look for

The output should simply be: Notebook is running. Do not move on until the cell runs without an error.

4. Check the Python version

The next cell imports **sys**, a small built-in Python module. It is not a data science package; it is just a way to ask Python about the running interpreter.

Why this cell exists

Version mismatches can cause confusing package behaviour. This cell records which Python version the notebook kernel is actually using.

Code cell

```
import sys

print(sys.version.split()[0])
```

What to look for

You should see a Python 3 version number. If this is not the environment you expected, switch the notebook kernel before continuing.

5. Check the required packages one at a time

These cells are setup checks, not a general permission to import everything at the top of every notebook. In the later steps, each import will still appear next to the first concept that needs it.

Package	Why it appears later
pandas	Creates and inspects tables of examples, features, and targets.
NumPy	Handles arrays, shapes, numeric operations, and model inputs.
matplotlib	Draws simple plots so model behaviour is visible.
scikit-learn	Provides train/test splitting, models, scalers, metrics, pipelines, and cross-validation.

Why these checks exist

These are setup checks. You are not using the packages for ML yet; you are just confirming that the notebook kernel can import the tools used later in the sequence.

Run the following as four separate notebook cells. Keeping them separate is useful because if one fails, the failing package is immediately obvious.

Cell 1 - check pandas - pandas is the table library used in Step 02.

Code cell

```
import pandas as pd
print("pandas", pd.__version__)
```

Cell 2 - check NumPy - NumPy supports arrays, shapes, and numeric operations.

Code cell

```
import numpy as np
print("NumPy", np.__version__)
```

Cell 3 - check matplotlib - matplotlib draws the simple plots used later.

Code cell

```
import matplotlib
print("matplotlib", matplotlib.__version__)
```

Cell 4 - check scikit-learn - scikit-learn provides the workflow helpers and models.

Code cell

```
import sklearn
print("scikit-learn", sklearn.__version__)
```

What to look for

Each cell should print a version number. A version number means that package is importable in the selected notebook kernel.

6. If a package is missing

Run installation commands in a terminal that is using the same environment as your notebook. If you are using the repo virtual environment, activate it first. The exact command may vary depending on your setup, but this is the standard pip form:

Terminal command

```
python -m pip install pandas numpy matplotlib scikit-learn jupyter
```

After installing

Restart the notebook kernel and re-run the package check cells. Installing into one environment while the notebook uses another is a common cause of confusion.

7. Working rules for the rest of the PDFs

The point of this sequence is not to produce a clever notebook quickly. The point is to make the ML workflow feel ordinary by building it in small, inspected steps.

- Run cells in order. If a later cell depends on an earlier one, do not jump ahead.
- Read the explanation immediately before the code cell, then run the code.
- After each output, ask: what changed, what shape is this, and what would I expect next?
- Prefer a boring cell you fully understand to a compact cell that hides five new ideas.
- If the output surprises you, stop and explain the surprise before continuing.

Notebook reset habit

At the end of each step, use Restart Kernel / Run All Cells. If the notebook only works because of some hidden state from earlier experimentation, it is not really under control.

8. Vocabulary preview

You do not need to memorise these yet, but they will appear repeatedly from Step 02 onward.

Word	Plain-English meaning for this sequence
sample	One example row, such as one house.
feature	An input column the model may use, such as bedrooms or floor area.
target / label	The answer column known during training, such as price.
X	The table or array of input features.
y	The target values we want the model to learn to predict.
fit	Train the model using X and y.
predict	Use the trained model to produce answers for new X rows.

Checkpoint before Step 02

You are ready for Step 02 when the notebook exists, the first print cell works, Python version is visible, and all four package checks print version numbers. Step 02 then starts the actual data work by creating a tiny house-price table row by row.

Terms to Say Out Loud

Step 01 - Get the Notebook Ready

Retrieval target

You do not need a dictionary definition. You should be able to say each term in plain English and connect it to the notebook work.

Term	Say it out loud as...
notebook	The working document where explanation, code cells, and outputs live together.
kernel	The running Python process that executes the notebook cells.
environment	The Python setup and installed packages the notebook is actually using.
package	An installed Python library, such as pandas, NumPy, matplotlib, or scikit-learn.
import	A line of code that makes a package, module, class, or function available.
version number	A printed package or Python version used to confirm what the notebook is running.
Restart Kernel / Run All Cells	The reset habit that checks the notebook works from a clean state.